Vellore Institute of Technology, Vellore

Computer Networks Lab (BCSE308P)

Review 2 – Research Project Progress Report

When Does Adapting Concurrency and Range Size Help?

A Controlled Study of Parallel HTTP Range Downloads

Submitted by

Name	Registration number
Sourajyoti Basak	24BCE0774
Shwetank Shrijan	24BDS0141
Ayush Konar	24BDS0107

October 2026

Contents

Abstract **1**

Terms and Abbreviations **2**

1 System Architecture **4**

1.1 Layers 4

1.2 Technology Stack 5

2 Detailed Design **6**

2.1 Range Conventions 6

2.2 Range Lifecycle 6

2.3 Controller Independence 6

2.4 Policy Interface 7

2.5 Configuration 7

3 Methodology and Algorithm **8**

3.1 Download Algorithm 8

3.2 Control Policies 8

3.3 Experimental Method 9

3.4 Paced Test Server 10

4 UML and Data-Flow Diagrams **11**

4.1 UML Class Diagram 11

4.2 UML State Diagram 12

4.3 Data-Flow Diagram (Level 1) 12

4.4 Run Record (Data Store D3) 13

5 Module Development **14**

6 Working Prototype **15**

6.1 Features 15

6.2 Prototype on the Live Internet 15

7 Implementation Progress **18**

7.1 Engineering Challenges Addressed 18

7.2 Planned for Review 3 18

8 Test Cases **19**

9 Initial Results **21**

9.1 Choosing the Setting 21

9.2 Why It Is Faster 21

9.3 Other Pacing Schedules 22

9.4 What This Result Does and Does Not Show 23

10 Individual Contribution **24**

References **25**

List of Figures

1.1 System architecture. One Downloader coordinates five parts; only the Fetcher talks to the network. 4

3.1 Algorithm flowchart for one worker (`internal/downloader/worker_pool.go`). 8
3.2 Reference pacing schedule D2. A request that starts in the first 4 s keeps its slow rate until it finishes. 10

4.1 UML class diagram of the main types. The Downloader owns its parts; every policy implements the same interface. 11
4.2 UML state diagram for one byte range in the RangeTracker. 12
4.3 Level-1 data-flow diagram of one download. 12

6.1 Throughput of a live download from `cdn.kernel.org` (8 connections, 1 MiB ranges), per second. 16
6.2 Terminal capture: live download and checksum verification. 16
6.3 Terminal capture: the automated test suite passing. 17

9.1 Completion time in seconds for all 40 (N,S) settings on the paced server. 21
9.2 Data received over time in one paired run, reconstructed from per-request server logs. . . 22
9.3 Mean completion time of ours and aria2 under the six pacing schedules S0–S5 defined in Table 3.3. 22

List of Tables

1.1 Technology stack 5

2.1 Components and their responsibilities 6

3.1 Experiment scenarios 9
3.2 Platform 9
3.3 Pacing schedules used in the sweep (rates per response; the “dip” is the 160 KiB/s phase) 10

4.1 Fields of the per-run JSON record 13

5.1 Modules and status 14

7.1 Milestones 18

8.1 Test types 19
8.2 Representative test cases 19
8.3 Verification commands and results 20

10.1 Team members 24

Abstract

Download managers speed up large downloads by fetching a file as several byte ranges over parallel HTTP connections. Two settings decide how well this works: the number of parallel connections, N, and the size of each requested range, S. This research project asks when changing N and S during a download actually helps. To answer it, we built an instrumented segmented HTTP range downloader in Go, whose N and S a control policy can change while a download is running, and a reproducible test platform around it: a local server with configurable rate pacing, Linux network emulation, and an automated benchmark against aria2, a widely used open-source download manager.

This report covers the system architecture, detailed design, algorithm, module development, working prototype, implementation progress, testing and initial results. In the first benchmark, on a 512 MiB file with server-side rate pacing, our downloader with 16 connections and 4 MiB ranges took 62.4 % less time than aria2 (time ratio 0.376), and was faster in 30 of 30 paired runs. Controls trace most of this gap to how request start times interact with per-response rate limiting, not to a general advantage over aria2. The project has 549 automated test functions, and all of them pass.

Terms and Abbreviations

Term	Meaning
N	Number of parallel connections, i.e. HTTP range requests in flight at once.
S	Range size: the number of bytes asked for in one request.
Range request	An HTTP request for part of a file (`Range: bytes=a-b`). A server that supports it answers with status `206 Partial Content` and a `Content-Range` header saying which bytes it sent.
Half-open range $[s, e)$	The bytes from s up to but not including e; its length is $e - s$. HTTP itself uses inclusive ranges.
aria2	A widely used open-source download manager that also downloads in parallel pieces; our external baseline, always run with `-x16 -s16 -k1M` (up to 16 connections, file split into 16 pieces, minimum piece 1 MiB).
`tc netem`	The Linux network emulator. We use it to add delay and packet loss to the test link.
Round trip (RTT)	Time for a packet to go to the server and back. Our emulated link has 80 ms.
KiB, MiB	Binary units: 1 KiB = 1,024 bytes, 1 MiB = 1,048,576 bytes. MB/s means 10^6 bytes per second.
Paced server	Our test server. It sends each response at a set rate, and a response keeps the rate in force when its request arrived.
Pacing schedule	The rate the paced server uses over time. D2 is the reference schedule: 640 KiB/s, a *dip* to 160 KiB/s from 2 s to 4 s, then 2,500 KiB/s.
Dip	The extra-slow phase in a schedule (160 KiB/s). "No dip" skips it; "long dip" makes it last until 8 s.
Paired run, block	Runs of every compared option under the same conditions, back to back in random order. A block is one such set.
Time ratio R	Our completion time divided by the other option's, averaged as a geometric mean over pairs. $R < 1$: ours took less time. $R = 0.376$ means 62.4 % less time.
Per-request and aggregate throughput	Per-request: how fast one request downloads. Aggregate: total bytes per second across all connections together.
Geometric mean	The average used for ratios: the exponential of the mean of the logarithms. It treats "twice as fast" and "half as fast" symmetrically.
95 % bootstrap interval	A range for R obtained by resampling the pairs 10,000 times. If it excludes 1, the difference is unlikely to be noise.
Screen	A small experiment (here 5 pairs) used to check a direction, not to confirm a result.
Protocol	The written plan of an experiment (runs, comparisons, decision rules), fixed before its first run.
Content-verified	The downloaded file was checked byte for byte against the source.

Term	Meaning
Policy, controller	A policy decides whether to change N or S; the controllers hold the current N and S targets within bounds. The policy runner passes each result to the policy and applies its decision.
Straggler splitting	An optional engine mode: when nothing is left to allocate, an idle connection takes over the unfinished second half of the range with the most bytes still to come.
LinUCB	A contextual bandit: a learning method that picks one of a few actions from the current measurements and learns from the reward that follows.
DQN	Deep Q-Network, a reinforcement-learning agent with a neural network that estimates the long-term value of each action. We use the Double DQN variant.
Mahimahi	A tool that replays recorded network traces, here cellular traces, so a download sees that network's real capacity over time.
Race detector	Go's built-in checker for unsafe concurrent access to shared memory, run on the test suite.

1. System Architecture

This is a research project: it studies when changing the number of connections N and the range size S during a download actually helps. The system below is the instrument for that study, a Go HTTP range downloader for one file from one server. It splits the file into byte ranges, fetches them over N parallel HTTP/1.1 connections, and lets a policy change N and the range size S during the download. Figure 1.1 shows the components and how they call each other.

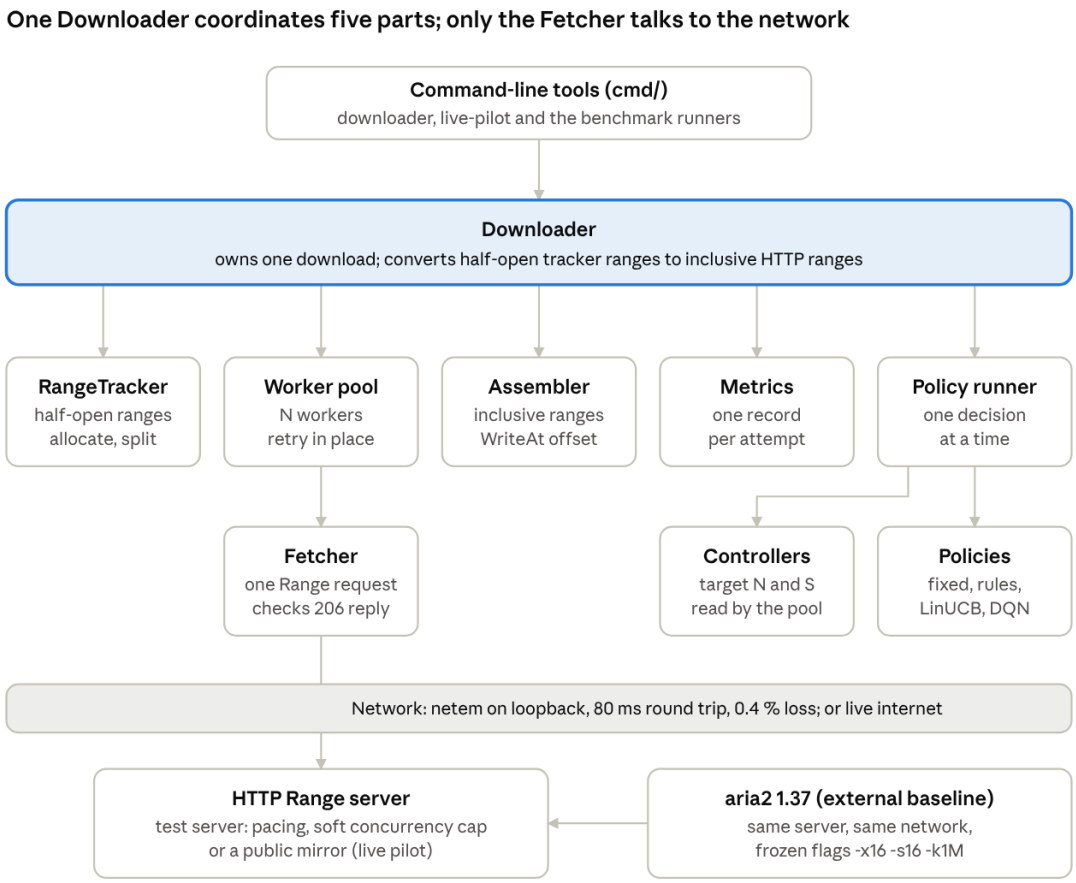


Figure 1.1: System architecture. One Downloader coordinates five parts; only the Fetcher talks to the network.

The worker pool asks the RangeTracker for the next range at the current size S, and the Fetcher downloads it. The Assembler writes it into the output file and Metrics records the attempt. After each attempt the policy runner asks the policy for an action and applies it to the controllers, which the pool reads on its next allocation.

1.1 Layers

- **Interface layer.** Command-line programs in `cmd/` parse flags and build a `downloader.Config`. The user-facing downloader and every experiment program use the same engine packages; the experiment programs also expose the policy and engine options.

- **Engine layer.** The Downloader and its five parts. Every part has one job and is tested on its own.
- **Control layer.** Controllers hold the current N and S targets, and policies decide changes. A policy only returns an action; it never touches the controllers directly.
- **Environment layer.** The HTTP server and the network. In experiments these are a local test server and Linux network emulation, so every run sees the same conditions.

1.2 Technology Stack

Table 1.1: Technology stack

Part	Technology	Purpose
Downloader and test server	Go 1.27 (`net/http`, HTTP/1.1)	Concurrency with goroutines
Linear algebra (LinUCB learner)	Gonum v0.17	Only external Go dependency
Network conditions	Linux `tc netem`	Delay and packet loss on the link
Recorded cellular traces	Mahimahi [4]	Replays real mobile traces
External baseline	aria2 1.37	Existing download manager
Training and analysis	Python 3, PyTorch 2.9, NumPy	DQN training, statistics

2. Detailed Design

The design rests on three rules: one HTTP request fetches one contiguous byte range, a range is never resized while it is downloading (an opt-in straggler-splitting mode is the one exception), and concurrency (N) and range size (S) are controlled separately. Table 2.1 lists the components.

Table 2.1: Components and their responsibilities

Component	Responsibility
RangeTracker	Splits the file into pending, in-flight and completed ranges
Fetcher	Sends one HTTP Range request and checks the `206` reply
Assembler	Writes each finished range at its offset in the output file
Worker pool	Runs N workers; a failed range is retried with the same bounds
Controllers and policy runner	Hold the N and S targets; apply one policy decision at a time
Metrics	Stores one record per request attempt

2.1 Range Conventions

The RangeTracker stores half-open ranges $[s, e)$, so a range's length is simply $e - s$. HTTP uses inclusive ranges [1], so the downloader converts at one point only:

$$[s, e) \longmapsto \texttt{Range: bytes=}s\texttt{-}(e-1) \tag{2.1}$$

For example, the tracker range $[0, 4194304)$ becomes the header `Range: bytes=0-4194303`. Keeping the conversion in one place removes the usual off-by-one errors.

2.2 Range Lifecycle

A range moves from *pending* to *in flight* when a worker allocates it, and to *completed* once its bytes are written (Figure 4.2). If a request fails, the worker retries the same range with the same boundaries, up to `MaxRetries` times. A range is never resized while it is in flight, so a change to S affects only ranges allocated later.

2.3 Controller Independence

Concurrency and range size are separate controls. Raising N starts more workers. Lowering N never cancels a running request: extra workers retire after their current range finishes. Changing S changes only the size of the next allocation.

2.4 Policy Interface

Every policy, rule-based or learned, implements the same Go interface, so one engine runs them all:

```
type Policy interface {
    // sees N, S, recent throughput, latency and request counts
    SelectAction(ctx Context) Action
    Update(ctx Context, action Action, reward float64)
}
// Actions: none, +/- concurrency, +/- chunk size
// (an action may carry a step count)
```

The policy runner holds a lock, so only one decision happens at a time, even when many requests finish together.

2.5 Configuration

One `downloader.Config` structure holds the URL, output path, file size, N, S, timeouts, the retry limit and optional minimum and maximum bounds for adaptive N. Invalid combinations are rejected before any request is sent.

3. Methodology and Algorithm

3.1 Download Algorithm

Each worker repeats one loop: take the next range of size S, download it, record the attempt, write it to the file, and let the policy adjust N or S (Figure 3.1).

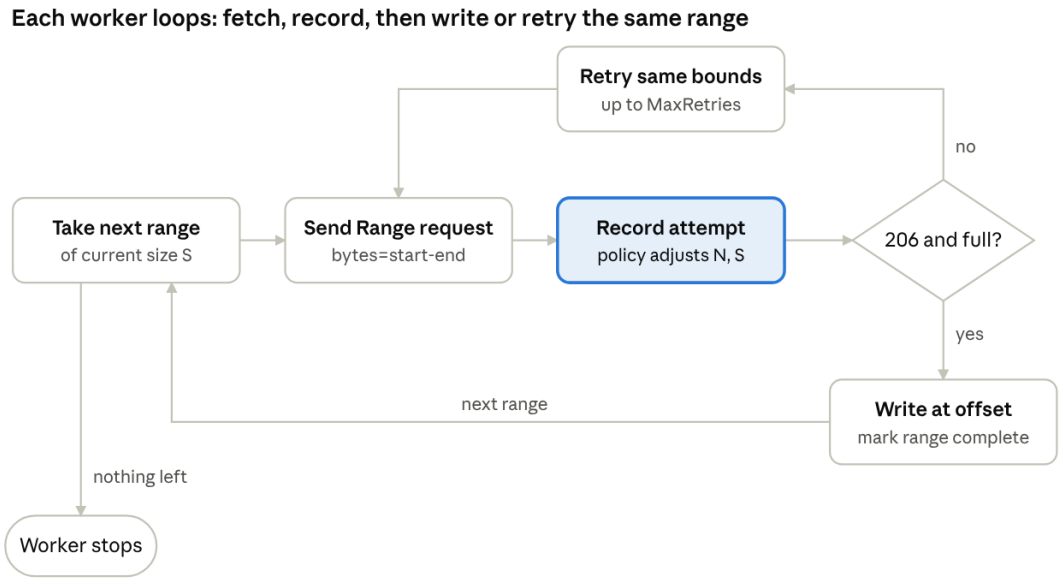


Figure 3.1: Algorithm flowchart for one worker (`internal/downloader/worker_pool.go`).

1. Read the current range size S and allocate the next range $[s, s+S)$ from the RangeTracker. If nothing is left, the worker stops.
2. Send `GET` with `Range:` `bytes=`s`-`$(s+S-1)$.
3. Record one metrics entry for the attempt and pass it to the policy runner, which may change N or S.
4. If the reply is `206` with the full length, write the bytes at offset s and mark the range complete. Otherwise retry the same range, up to `MaxRetries` times.

3.2 Control Policies

The policies compared are fixed (N,S) settings, simple throughput rules that step N or S up or down, and aria2 1.37 as the external baseline. After each request the throughput rule computes the recent throughput T_{now} over the last 8 attempts, and compares it with T_{prev}, the value at the previous decision:

$$\Delta = \frac{T_{\text{now}} - T_{\text{prev}}}{T_{\text{prev}}}, \qquad \text{action} = \begin{cases} \text{increase } N \text{ (or } S\text{)} & \text{if } \Delta \geq \theta_{\text{up}} \\ \text{decrease } N \text{ (or } S\text{)} & \text{if } \Delta \leq -\theta_{\text{down}} \\ \text{no change} & \text{otherwise} \end{cases} \tag{3.1}$$

In our runs both thresholds were 10 % ($\theta_{\text{up}} = \theta_{\text{down}} = 0.10$). The joint policy alternates between changing N and changing S, and switches to the other dimension when one reaches its bound.

3.3 Experimental Method

Experiments use a local HTTP server with configurable rate limiting, and Linux `tc netem` [3] adding an 80 ms round trip and 0.4 % packet loss. Every run uses a fresh server and output file. Table 3.1 lists the scenarios.

Table 3.1: Experiment scenarios

Scenario	Server behaviour	What it tests
Paced (D2)	640 KiB/s per response, 160 KiB/s from 2 s, 2,500 KiB/s from 4 s	Timing of request starts against rate changes
Pacing sweep	Six variants of the schedule above	Whether the result depends on one schedule
Stock NGINX	`limit_rate` per response	A real web server instead of our test server

Table 3.2: Platform

Item	Configuration
Host (project machine)	AMD Ryzen 5 5500U laptop (6 cores, 12 threads); client and test server on the same machine
Operating system	Ubuntu 24.04 container on a Linux 7.x kernel
Software	Go 1.27.1, aria2 1.37.0, Python 3.14
Network emulation	`tc netem` on loopback: 40 ms each way (80 ms round trip), 0.4 % loss each way
Test file	512 MiB, deterministic content, verified byte for byte
aria2 settings	`-x16 -s16 -k1M`, the same in every run

Baseline. We compare against aria2 [2] because it is the standard open-source segmented downloader on Linux; IDM runs only on Windows. Its flags were chosen once, from a discovery sweep of its split settings, and kept the same in every run.

Each pair runs our downloader and aria2 back to back in random order. With t_i the completion time in pair i, the result is the geometric mean of the per-pair time ratios,

$$R = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \ln \frac{t_i^{\text{ours}}}{t_i^{\text{aria2}}}\right), \tag{3.2}$$

reported with a 95 % percentile bootstrap interval from 10,000 resamples. $R < 1$ means ours was faster. Each protocol is written down before its first run, later amendments are dated and disclosed, and every run

is kept, including failures.

3.4 Paced Test Server

The main test server is written in Go and is part of this project. It serves a 512 MiB file with deterministic content, honours HTTP Range requests, and paces every response at a set rate. Three rules define its behaviour:

1. Each response is sent at the server's rate *at the moment the request arrives*, and keeps that rate until the response finishes.
2. A schedule changes the rate at fixed times. A change affects only requests that start afterwards, not ones already running.
3. Rates are per response, with no shared limit, so opening more connections adds bandwidth.

This makes the server a controlled stand-in for per-connection rate limiting, such as NGINX's `limit_rate`, combined with a rate change during the download. Because the rule is explicit, any result can be traced back to it. Figure 3.2 shows the reference schedule, D2.

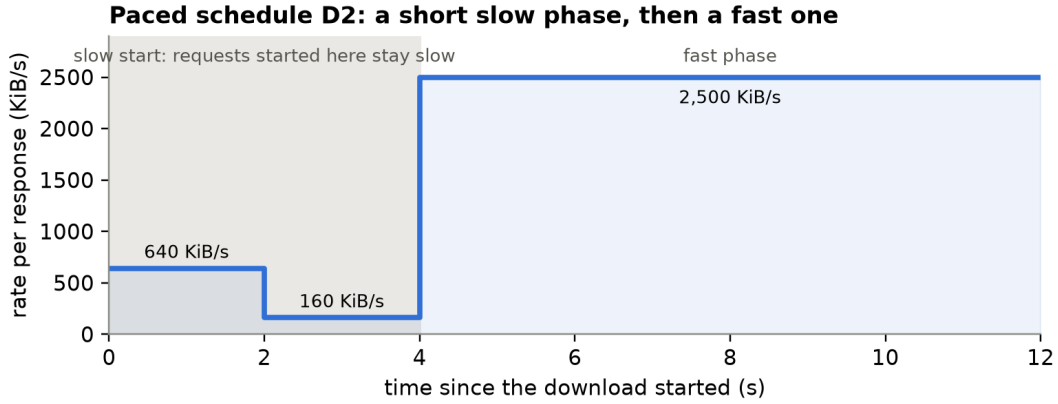


Figure 3.2: Reference pacing schedule D2. A request that starts in the first 4 s keeps its slow rate until it finishes.

Table 3.3: Pacing schedules used in the sweep (rates per response; the "dip" is the 160 KiB/s phase)

Schedule	Rates over time	What it varies
S0 D2 reference	640 KiB/s; 160 from 2 s; 2,500 from 4 s	Reference: short slow phase
S1 No dip	640 KiB/s; 2,500 from 2 s	No extra-slow dip
S2 Long dip	640 KiB/s; 160 from 2 s; 2,500 from 8 s	Dip lasts 6 s instead of 2 s
S3 Partial recovery	640 KiB/s; 160 from 2 s; 1,280 from 4 s	Recovery to half speed
S4 Constant fast	2,500 KiB/s throughout	No rate change
S5 Constant slow	640 KiB/s throughout	No rate change

The same server can also switch on a soft concurrency cap and inject faults, such as error statuses or failing ranges. Those features are used in later experiments and in the failure-handling tests.

4. UML and Data-Flow Diagrams

The project has no database. Download state lives in memory, and results are written as JSON and JSONL files under `testdata/`, one record per run.

4.1 UML Class Diagram

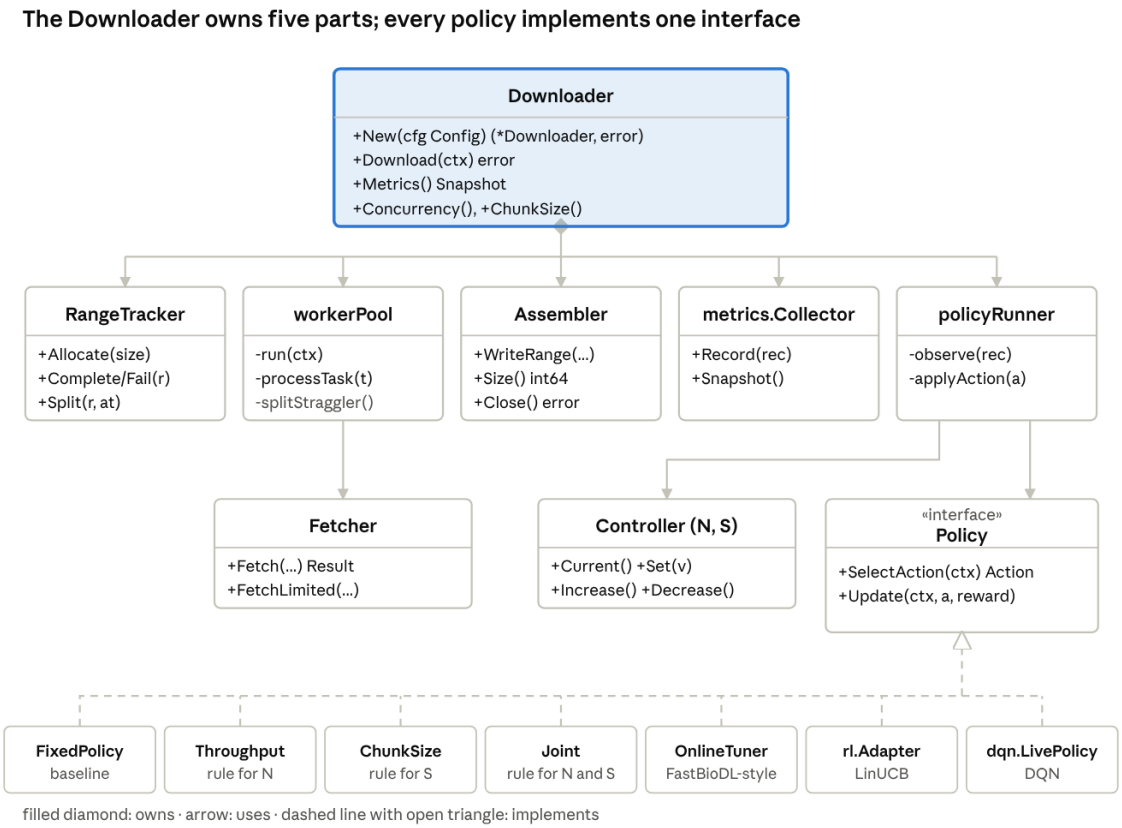


Figure 4.1: UML class diagram of the main types. The Downloader owns its parts; every policy implements the same interface.

Only the policy runner calls a policy, and a policy changes nothing directly: it returns an action that the runner applies to the controllers.

4.2 UML State Diagram

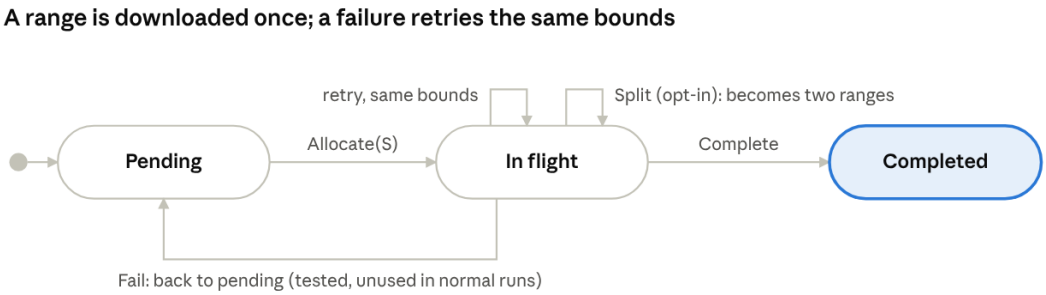


Figure 4.2: UML state diagram for one byte range in the RangeTracker.

A range never changes size while in flight. The only exception is an opt-in split, which turns one range into two. The tracker also supports returning a failed range to pending, which is tested, but the downloader retries a failed range in place instead, so its boundaries never change.

4.3 Data-Flow Diagram (Level 1)

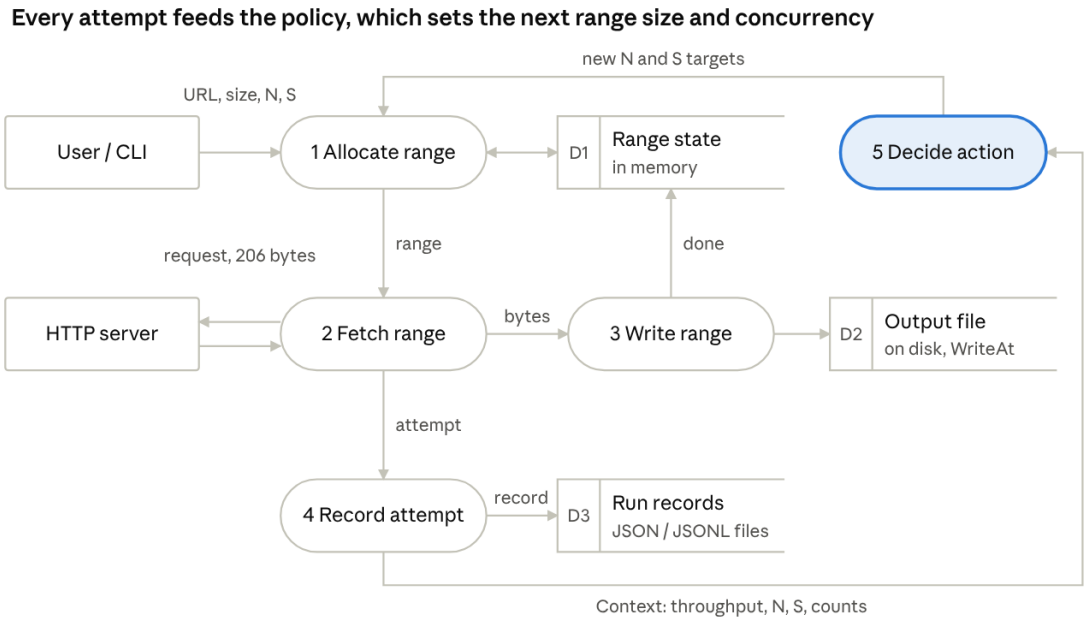


Figure 4.3: Level-1 data-flow diagram of one download.

The loop on the right is the adaptive part: each attempt's result reaches the policy, which can change N and S for the next allocations. D1–D3 are the data stores: range state in memory, the output file, and the run records.

4.4 Run Record (Data Store D3)

Each run writes one JSON record (Table 4.1). The analysis scripts read only these files, so every reported number can be traced to a run.

Table 4.1: Fields of the per-run JSON record

Field	Meaning
`run_id`, `system`, `config`	Which run, which downloader (ours or aria2), and its N and S
`completion_ms`	Wall-clock time to finish the download
`output_sha256`, `content_verified`	Checksum of the output, and whether it matches the source
`coverage_no_missing`, `coverage_no_duplicates`	Every byte fetched exactly once
`netem_status`	Whether the network emulation was applied successfully
`server_requests`	Per-request start, end, rate and byte range from the server log
`client_requests`, `client_retries`	Request counts seen by the downloader

5. Module Development

Table 5.1: Modules and status

Module	Purpose	Status
`rangetracker`, `fetcher`, `assembler`	Core download engine	Done
`downloader`	Worker pool, policy runner, coordination	Done
`concurrency`, `chunksize`, `policy`	Controllers and rule-based policies	Done
`metrics`	Per-attempt measurements	Done
`experiment`	Test server and netem control	Done
`rl`	Learned (RL) policies	Implemented; evaluated in Review 3

The modules depend on each other in this order:

1. **RangeTracker** is the base, because every other part depends on its rules. It keeps pending ranges sorted and merges neighbours, and a single lock makes it safe for many workers at once.
2. **Fetcher** sends one Range request and rejects any reply that is not a `206` of the expected length, or whose `Content-Range`, when present, does not match. It uses a dedicated HTTP/1.1 transport, so each connection carries exactly one request at a time.
3. **Assembler** writes each range with `WriteAt` at its own offset, so ranges can finish in any order. The file is sized to its final length up front.
4. **Worker pool and Downloader** connect the parts. A manager goroutine starts and retires workers to match the target N, and every attempt is recorded exactly once, whether it succeeds or fails.
5. **Controllers and policies** sit on top of the fixed engine and never touch it directly. Each controller clamps its target to configured bounds.
6. **Experiment platform:** a test server with configurable per-response pacing, `tc netem` control that records whether the shaping actually applied, and runners that write one JSON record per run.

Each module has its own tests, and the whole suite also runs under Go's race detector.

6. Working Prototype

The downloader runs as a command-line tool and produces a byte-identical copy of the file. It first checks that the server supports Range requests and reads the file size, and it can verify the result against an expected SHA-256:

```
go build ./...
go run ./cmd/downloader -url <URL> -out file.bin \
    -concurrency 8 -chunk 1048576 -sha256 <expected hash>
```

The program `cmd/aria2-benchmark` runs our downloader and aria2 on the same test server for a side-by-side comparison.

6.1 Features

- Downloads a single file from one server over HTTP/1.1 Range requests, using N parallel connections (resume, authentication and HTTP/2 are out of scope).
- The engine lets a policy change N and S during the download without cancelling any running request (used through the experiment programs).
- Checks every reply's status (`206`), length and, when present, `Content-Range`, and retries failed ranges with the same bounds.
- Writes ranges straight to their offsets, so the finished file is byte-identical to the source.
- Records one measurement per request attempt for later analysis.

6.2 Prototype on the Live Internet

Figure 6.1 shows the prototype downloading the Linux 3.0 source archive from `cdn.kernel.org`. The file's SHA-256 matched the checksum published by kernel.org. The terminal capture below is a separate run of the same download, so its time differs slightly.

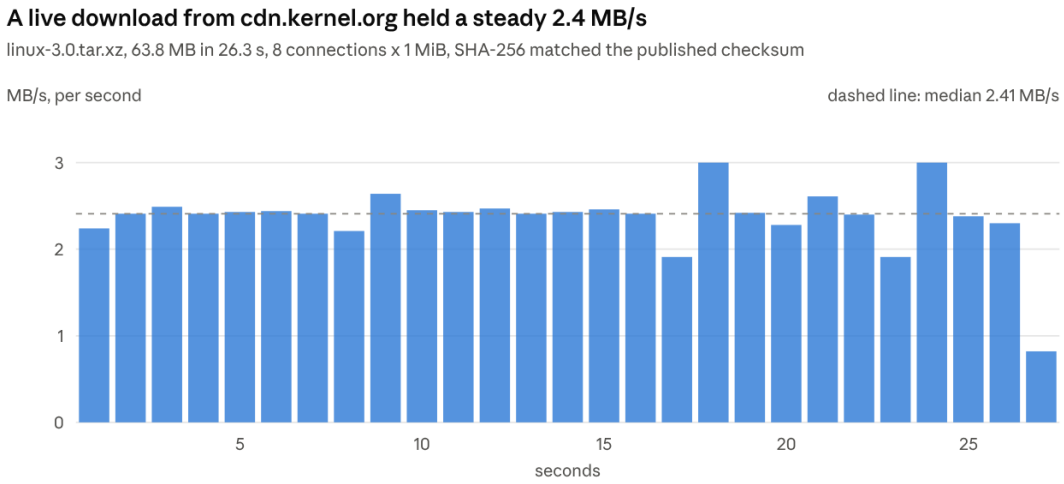


Figure 6.1: Throughput of a live download from `cdn.kernel.org` (8 connections, 1 MiB ranges), per second.

Live download from cdn.kernel.org with checksum verification

```
$ ./downloader -url https://cdn.kernel.org/pub/linux/kernel/v3.x/linux-3.0.tar.xz -out
    linux-3.0.tar.xz -concurrency 8 -chunk 1048576 -sha256
    ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6
server supports range requests; size 63798008 bytes
downloaded 63798008 bytes to linux-3.0.tar.xz in 26.7 s (2.39 MB/s)
sha256 OK ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6
$ curl -s https://cdn.kernel.org/pub/linux/kernel/v3.x/sha256sums.asc | grep linux-3.0.tar.xz
ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6  linux-3.0.tar.xz
$ sha256sum linux-3.0.tar.xz
ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6  linux-3.0.tar.xz
```

Figure 6.2: Terminal capture: live download and checksum verification.

Automated test suite: every package passes

```
$ go test -p 1 ./...    (packages with tests shown)
ok      adaptivedl/cmd/downloader       0.018s
ok      adaptivedl/cmd/live-pilot       0.097s
ok      adaptivedl/internal/assembler   (cached)
ok      adaptivedl/internal/chunksize   (cached)
ok      adaptivedl/internal/concurrency (cached)
ok      adaptivedl/internal/diagnostics 1.247s
ok      adaptivedl/internal/downloader  1.733s
ok      adaptivedl/internal/experiment  20.824s
ok      adaptivedl/internal/experiment/epoch    (cached)
ok      adaptivedl/internal/experiment/network  (cached)
ok      adaptivedl/internal/experiment/pilot    (cached)
ok      adaptivedl/internal/experiment/replay   28.331s
ok      adaptivedl/internal/experiment/server   (cached)
ok      adaptivedl/internal/experiment/tau      (cached)
ok      adaptivedl/internal/fetcher     0.655s
ok      adaptivedl/internal/metrics     0.003s
ok      adaptivedl/internal/policy      0.008s
ok      adaptivedl/internal/rangetracker        (cached)
ok      adaptivedl/internal/rl  2.350s
ok      adaptivedl/internal/rl/dqn      0.012s
$ grep -c FAIL race-detector-run.log
0
```

Figure 6.3: Terminal capture: the automated test suite passing.

7. Implementation Progress

All planned modules are built and tested. The work so far covers the engine, the control layer, the test platform and the first benchmark against aria2 (Table 7.1).

Table 7.1: Milestones

Milestone	Status
Core engine, controllers, rule-based policies	Done
Test server, network emulation, benchmark harness	Done
Baseline benchmark against aria2 (30 paired runs)	Done
Pacing-schedule screen and stock NGINX check	Done
Recorded cellular-trace replay platform (Mahimahi)	Implemented; evaluated in Review 3
Learned policies (LinUCB, DQN)	Implemented; evaluated in Review 3

7.1 Engineering Challenges Addressed

- **Off-by-one range errors:** avoided by doing the half-open to inclusive conversion in a single place.
- **Retry semantics:** a failed range is retried with its original boundaries, never re-split, so accounting and offsets stay stable.
- **Silent network-emulation failures:** each run records whether `tc netem` actually applied; runs where it did not are excluded and reported.
- **Fair comparisons:** both downloaders get the same file, the same server, a fresh output, and a random order within each pair.

7.2 Planned for Review 3

- Adaptive control under a server whose speed changes during a download.
- Results of the learned policies (LinUCB, DQN).
- Recorded cellular-trace replay and a live-internet study on public mirrors.
- Performance analysis and the full comparison with aria2.

8. Test Cases

The project has **549 automated test functions in 29 Go packages**, and all of them pass. No test needs internet access: each one starts its own local HTTP server, so results do not depend on outside conditions. Table 8.1 summarises the test types.

Table 8.1: Test types

Type	What it covers	Examples
Unit	One module alone: allocation, reply checks, file writes, controller bounds	RangeTracker, Fetcher
Integration	Full download against a local server, then a byte-for-byte check	Downloader, 1 to *N* workers
Failure handling	Server errors, short bodies, wrong headers, `200` instead of `206`	Fetcher, retry tests
Concurrency	Many workers at once, under Go's race detector	Worker pool, policy runner
Policy	Decisions for rising, falling and flat throughput, and at bounds	Throughput, joint policies
Experiment platform	Server pacing, netem commands, run-record format	`internal/experiment`

Table 8.2: Representative test cases

No.	Test case	Expected result	Status
1	Allocate a 100-byte file in 10-byte ranges	Ranges in order, none overlapping	Pass
2	Server ignores Range and returns `200`	Request rejected as an error	Pass
3	Body shorter than the requested range	Request rejected as an error	Pass
4	Ranges written out of order	Output file is byte-identical	Pass
5	First attempt fails, second succeeds	Same range retried; file correct	Pass
6	Range keeps failing	Download stops after the retry limit	Pass
7	Policy lowers *N* mid-download	No request is cancelled	Pass
8	Many workers finish at once	Policy decisions run one at a time	Pass

Table 8.3: Verification commands and results

Command	Result
`go build ./...`	Builds cleanly
`go vet ./...`	No issues
`gofmt -l internal cmd`	No unformatted files
`go test ./...`	All 549 test functions pass
`go test -race ./...`	Pass; no data races found
`go test -count=5 ./...`	Pass on 5 repeated runs

In one earlier repeated run, the trace-replay test `TestAnchorCalibration` failed once under heavy machine load. It passed in all 11 reruns and is tracked as load-sensitive rather than hidden.

9. Initial Results

On a 512 MiB file with server-side rate pacing, our downloader with 16 connections and 4 MiB ranges took **62.4 % less time than aria2**: it was faster in 30 of 30 paired runs, with a time ratio of $R = 0.376$ (95 % interval [0.373, 0.379]). These 30 pairs were new runs, separate from the selection grid below.

9.1 Choosing the Setting

We first ran all 40 combinations of N and S on the paced server (Figure 9.1). More connections always helped, and at every N a 4 MiB range size was fastest or tied for fastest. We stopped at $N = 16$ to match aria2's 16-connection setting. Each cell is one discovery run.

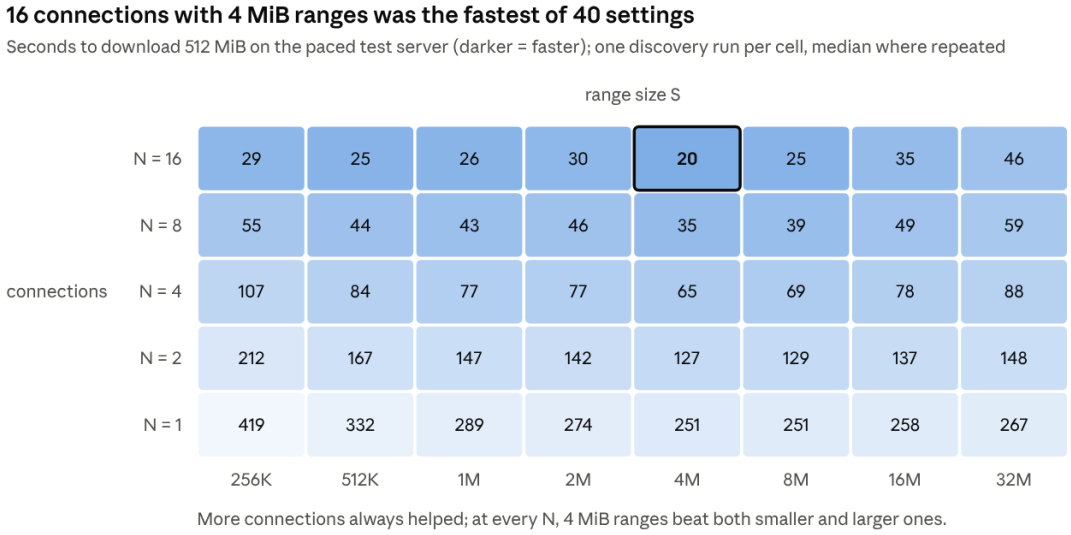


Figure 9.1: Completion time in seconds for all 40 (N, S) settings on the paced server.

9.2 Why It Is Faster

The server sends slowly for the first 4 seconds, then speeds up, and each request keeps the rate it started with. aria2 starts all its pieces within the first 2 seconds, so they stay slow. Our smaller ranges keep starting new requests after the speed-up (Figure 9.2).

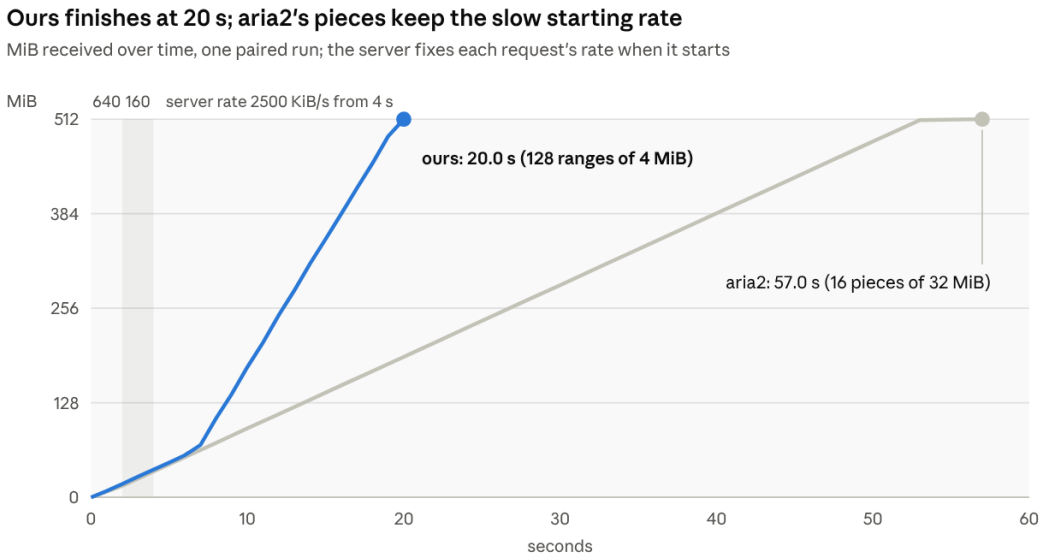


Figure 9.2: Data received over time in one paired run, reconstructed from per-request server logs.

9.3 Other Pacing Schedules

A screening run over six pacing schedules (5 pairs each, 60 runs, all content-verified) found ours faster in 28 of 30 pairs (Figure 9.3). The two were level only when the server sent at one constant slow rate.

A server we did not write. To check that the direction does not depend on our own server, we repeated the comparison against stock NGINX 1.24 in two configurations: `limit_rate 640k` per response, and the same with the first 1 MiB of each response unthrottled (`limit_rate_after 1m`). Ours was faster in 10 of 10 pairs ($R = 0.863$ and 0.597). With 5 pairs per configuration, this is a screen, not a confirmation.

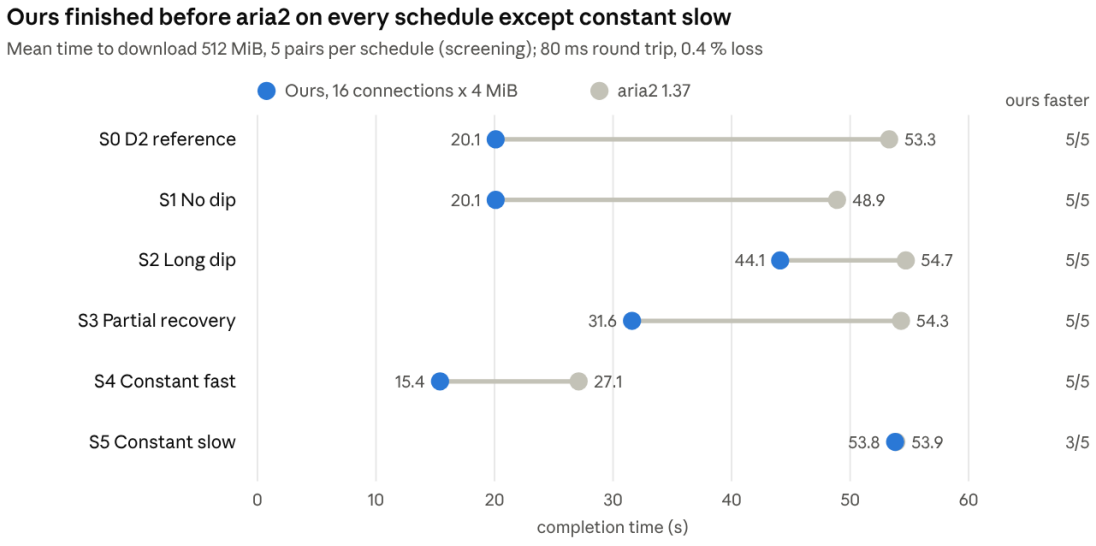


Figure 9.3: Mean completion time of ours and aria2 under the six pacing schedules S0–S5 defined in Table 3.3.

9.4 What This Result Does and Does Not Show

This is not evidence that our downloader is better than aria2 in general. The gap comes from one specific interaction:

- The test server fixes each response's rate when the request starts. A request that starts during the slow phase stays slow until it finishes.
- With 16 connections, aria2 splits a 512 MiB file into pieces of about 32 MiB and starts them all within the first 2 seconds. In a separate side check, outside the benchmark, raising its `--split` setting did not give smaller first pieces.
- With 4 MiB ranges, 112 of our 128 requests started after the server sped up.

Two controls support this reading. When our downloader used aria2-like piece sizes, the gap shrank from $R = 0.376$ to $R = 0.862$ [0.834, 0.890]. At a constant slow rate the two were level ($R = 0.998$). Not all of the gap is explained: ours stayed 14 % faster with aria2-like pieces, and was also faster at a constant fast rate with no rate change (15.4 s against 27.1 s). We did not isolate that residual. The fair conclusion is narrower: *on servers that rate-limit per response, smaller ranges that start more requests over time finish sooner than aria2's large initial pieces.* These results come from one machine with emulated network conditions; Review 3 adds a live-internet study and recorded cellular traces. The next stage tests whether adaptive control adds anything beyond a well-chosen fixed setting.

10. Individual Contribution

Table 10.1: Team members

Member	Reg. no.	Contribution
Sourajyoti Basak	24BCE0774	Downloader architecture and core implementation; adaptive concurrency and range management; test server and Linux network experiments; system integration and validation
Shwetank Shrijan	24BDS0141	Downloader development and testing of assigned components; monitoring and experiment automation; baseline and heuristic evaluation
Ayush Konar	24BDS0107	LinUCB; RL state, action and reward design; Python/PyTorch models; integration of the learned policy with the Go downloader; ML/RL evaluation

Responsibilities follow the division of work set out in the Review 1 proposal.

References

[1] R. Fielding, M. Nottingham and J. Reschke, “HTTP Semantics,” RFC 9110, IETF, 2022. (Range requests, Section 14.)

[2] aria2 project, “aria2: The ultra fast download utility,” version 1.37. `https://aria2.github.io/`

[3] Linux Foundation, “netem: Network Emulator,” `tc-netem`(8) manual page.

[4] R. Netravali et al., “Mahimahi: Accurate Record-and-Replay for HTTP,” USENIX ATC, 2015.

[5] The Go Authors, “The Go Programming Language,” `https://go.dev/`